

استاد: دکتر داوودآبادی

نام درس: مبانی پردازش زبان و گفتار

گزارش: HW1

نام و نام‌خانوادگی: نیوشا یقینی

شماره دانشجویی: ۹۸۵۲۲۳۴۶

فروردین ۱۴۰۴

فهرست مطالب

۱. سوال شماره ۱ ..... ۳
۲. سوال شماره ۲ ..... ۶
۳. سوال شماره ۳ ..... ۱۲
۴. سوال شماره ۴ ..... ۱۴
۵. سوال شماره ۵ ..... ۲۰
۶. سوال شماره ۶ ..... ۲۹
منابع ۳۶

۱. سوال شماره ۱:

The quick brown fox jumps over the lazy dog. The dog barked, and the fox ran away. Meanwhile, another fox appeared near the forest, and walked in a level path led to the lake.

۱,۱ یک الگوی regex بنویسید تا همه موارد fox یا dog را در متن پیدا کند.

از آنجایی که می‌خواهیم کلمات fox یا dog بصورت انحصاری پیدا شوند از الگوی \b در قبل و بعد آنها استفاده می‌کنیم تا کلمات را بدون اینکه به کاراکتر دیگری چسبیده باشند پیدا کند. \b یک **مرز کلمه (word boundary)** هست و تمام حالت‌های دیگر را به جز کاراکتر می‌پذیرد، مثل:

: - “ newline tab () whitespace ! ? ,

\b(fox|dog)\b

The quick brown fox jumps over the lazy dog. The dog barked, and the fox ran away. Meanwhile, another fox appeared near the forest, and walked in a level path led to the lake.

با کمک الگوی بالا کلمات هایلایت شده پیدا می‌شوند.

در نظر گرفتیم که نیازی به case-insensitive بودن نداریم، اگر بخواهیم از این حالت استفاده کنیم این الگو را استفاده می‌کنیم:

(?i)\b(fox|dog)\b

۲,۱ الگوی regex سوال قبل را طوری تغییر دهید تا کلماتی که با f یا d شروع می‌شوند و با x یا g پایان می‌یابند (مثلاً fox یا dog) را بیابد.

برای این مثال می‌توانیم از الگوی زیر استفاده کنیم، این الگو برای ابتدای کلمات d یا f را در نظر می‌گیرد و سپس حروف دیگر را به تعداد صفر یا بیشتر می‌پذیرد، و پس از آن کلمه باید با x یا g پایان یابد.

\b[fd][a-zA-Z]*[xg]\b

نکته اینجاست که به جای قسمت [a-z, A-Z]* می‌توانیم از \w* هم استفاده کنیم، این الگو حالت‌های کاراکتر را شامل می‌شود که برابر زیر است:

a-z, A-Z, 0-9, _

\b[fd]\w*[xg]\b

نکته دیگر این است که در این مثال می‌توانیم به جای [fd] از (f|d) هم استفاده کنیم و تفاوتی ایجاد نمی‌کند، منتها برای تک کاراکترها استفاده از [fd] مناسب‌تر است. در این مثال هم تمام حالت‌های مثال قبل پیدا می‌شوند، البته مثال هایی از قبیل زیر هم اگر وجود داشت پیدا می‌شدند:

fox, fix, fog, flag, firing, dog, dig, drag, dox, ...

۳٫۱ یک الگوی regex بنویسید تا همه کلماتی که حداقل دو حرف صدادار دارند (مثلاً over یا barked) را پیدا کند.

Vowels: a, e, i, o, u

الگوی زیر مشخص می‌کند، که در هر جای یک کلمه مستقل می‌تواند حروف صدادار وجود داشته باشند، و محدودیتی در position آنها در جمله وجود ندارد، اما در تمام حالت‌های کلمه حداقل ۲ بار این حروف صدادار باید دیده شوند، و در صورتی که بیشتر هم دیده شوند در دسته \w قرار می‌گیرند و محدودیتی برایشان وجود ندارد.

\b\w*[aeiou]\w*[aeiou]\w*\b

The quick brown fox jumps over the lazy dog. The dog barked, and the fox ran away. Meanwhile, another fox appeared near the forest, and walked in a level path led to the lake.

با کمک الگوی بالا کلمات هایلایت شده پیدا می‌شوند.

۴٫۱ یک الگوی regex بنویسید تا کلماتی که با یک حرف شروع و با همان حرف پایان می‌یابند را بیابد.

در الگوی زیر، با استفاده از ([a-zA-Z]) اولین کاراکتر از یک کلمه مستقل را در گروه ۱ ذخیره می‌کنیم در ادامه کاراکترهای دیگری به تعداد صفر یا بیشتر می‌توانند وجود داشته باشند و سپس شرط وجود کاراکتر گروه ۱ را در انتهای کلمه اعمال می‌کنیم.

\b([a-zA-Z])[a-zA-Z]*\1\b

The quick brown fox jumps over the lazy dog. The dog barked, and the fox ran away. Meanwhile, another fox appeared near the forest, and walked in a level path led to the lake.

کلمه هایلایت شده در بالا، تنها کلمه‌ای است که این الگو پیدا می‌کند.

۵٫۱ یک الگوی regex بنویسید تا همه کلماتی که با حرف l شروع می‌شوند و در انتهای جمله قرار دارند را پیدا کند.

ابتدا در نظر می‌گیریم که کلمه‌ای که مدنظر داریم با حرف l به صورت lowercase یا L به صورت uppercase شروع می‌شود.

سپس در نظر می‌گیریم که شرط اینکه کلمه‌ای در انتهای جمله باشد، این است که بعد از آن یکی از علامت‌های نقطه، علامت سوال یا علامت تعجب دیده شود، و پس از آن نیز یا فاصله داشته باشیم که یعنی یک جمله در اواسط متن بوده، یا به انتهای string (کل متن) رسیده باشیم.

قابل ذکر است که \s شامل انواع spaceها می‌شود، مثل:

Whitespace, tab, newline, ...

نکته بعدی این است که دقت شود در این قسمت (\s|$) نمی‌توان از حالت دیگری مثل [\s$] استفاده شود، چرا که از characterها فقط در [] استفاده می‌کنیم، و اینها در داخلش به عنوان کاراکتر در نظر گرفته می‌شوند.

نکته دیگر این است که اگر بخواهیم در حالت ساده در نظر بگیریم می‌توانیم از الگوی زیر استفاده کنیم:

\b[Ll][a-zA-Z]*[.!?](\s|$)

اما باید دقت کنیم که استفاده از این الگو باعث می‌شود که کلمه‌ای مثل "lake" در انتهای جمله به درستی match نشود، چرا که این الگو سعی می‌کند کلمه ای را با ویژگی‌های زیر انتخاب کند:

- کلمه، به علاوه‌ی علامت های نگارشی ذکر شده، به علاوه‌ی فاصله

و در آن الگو کلمه به این حالت انتخاب می‌شود = "lake."

در صورتی که ما می‌خواهیم کلمه بصورت مستقل match شود، و علامت‌های نگارشی بیرون از کلمه درنظر گرفته شوند بنابراین برای اینکار از lookahead استفاده می‌کنیم. از lookahead برای چک کردن آنچه که بعد از یک کلمه می‌آید استفاده می‌کنیم، اما آن چیزی که بعد از کلمه (مانند نقطه، علامت تعجب یا علامت سوال) در **خود match** نخواهد شد.

با الگوری زیر با استفاده از ((?=[.!?](\s|$)) در regex به این شکل عمل می‌کنیم که کلمه‌ای را پیدا کرده، سپس بررسی می‌کنیم (نگاه می‌کنیم) که آیا بعد از آن نقطه‌گذاری مانند نقطه یا علامت تعجب، یا پایان رشته آمده است یا نه اما از آن استفاده نمی‌کنیم.

\b[lL][a-zA-Z]*\b(?=[.!?](\s|$))

بنابراین کلمه ذکر شده در الگوی جدید به صورت مستقل انتخاب می‌شود = "lake"

The quick brown fox jumps over the lazy dog. The dog barked, and the fox ran away. Meanwhile, another fox appeared near the forest, and walked in a level path led to the lake.

کلمه هایلایت شده در بالا، تنها کلمه‌ای است که این الگو پیدا می‌کند.

۲. سوال شماره ۲:

Stemming و Lemmatization را توضیح دهید، سپس تفاوت‌های آن‌ها را با مثالی در زبان فارسی بیان کنید.

1) Stemming:

Stemming یک فرآیند الگوریتمی است که برای کاهش کلمات به ریشه‌شان استفاده می‌شود. در این روش، پسوندها، پیشوندها، و سایر تغییرات دستوری از کلمه حذف می‌شود تا کلمه به ریشه‌اش (که ممکن است معنای دقیقی نداشته باشد) برسد. Stemming هیچ‌گونه دقت معنایی ندارد و صرفاً براساس قواعد الگوریتمی و می‌تواند باعث کاهش دقت در تشخیص معانی صحیح بشود.

مزایا و معایب:

- **مزایا**:
  - دقت بالا در حفظ معنای کلمات
  - استفاده در کاربردهای پیچیده‌تری چون تحلیل‌های معنایی و ترجمه ماشینی
- **معایب**:
  - زمان‌بر و پیچیده‌تر از Stemming
  - نیاز به منابع دیکشنری و اطلاعات گرامری دقیق

مثال در زبان فارسی:

- کتاب → کتاب‌ها
- دوید → دویدن

در این مثال‌ها پسوندها حذف شده‌اند و کلمه به شکل ریشه‌اش تبدیل شده است.

در Stemming، معمولاً بیشتر تمرکز بر روی حذف **پسوندها** است اما گاهی بعضی از الگوریتم‌ها برای زبان‌های خاص (مثل زبان‌های اروپایی یا زبان‌های هندواروپایی) ممکن است کمی پیشوندها را نیز در نظر بگیرند.

معروف‌ترین الگوریتم‌ها:

الگوریتم‌های Stemming اصولاً به حذف پسوندها به صورت سیستماتیک و الگوریتمی می‌پردازند. برخی از معروف‌ترین الگوریتم‌ها عبارتند از:

- Porter Stemmer:
  - معروف‌ترین الگوریتم، متمرکز بر حذف پسوندها مثل "ing-" و."ed-"
  - استفاده از قوانین مشخص برای کاهش کلمات.

- Snowball Stemmer (Porter 2):
  - نسخه بهبود یافته Porter با قوانین بهینه‌تر و دقت بیشتر.
- Lancaster Stemmer:
  - تهاجمی‌تر و شدیدتر از Porter و Snowball؛ کلمات را بیش از حد کوتاه می‌کند.
- Lovins Stemmer:
  - اولین الگوریتم Stemming، دقیق‌تر از Porter اما طراحی‌شده برای زبان انگلیسی.

برای این روش در زبان فارسی نیز الگوریتم‌هایی وجود دارند که طراحی شده‌اند تا پسوندها و پیشوندهای زبان فارسی را حذف کنند و به ریشه کلمات برسند:

- Hazm:
  - کتابخانه معروف برای NLP فارسی؛ بیشتر روی حذف پسوندها تمرکز دارد.
  - پیشوندها معمولاً حذف نمی‌شوند چون حذف آن‌ها به معنای کلمه آسیب می‌زند.
  - مثال پسوندها: "-ها"، "-ی"، "-ترین".
- AlmaStem:
  - مخصوص NLP فارسی برای کاهش حجم داده‌ها و گروه‌بندی کلمات مشابه.
- Kashgari:
  - ابزار NLP با روش‌های پیشرفته، پشتیبانی از Stemming برای زبان‌های مختلف از جمله فارسی.

کاربردها و استفاده‌ها:

- **تحلیل متن**: در تحلیل‌های متن برای شناسایی کلمات مشابه و کاهش حجم داده‌ها از Stemming استفاده می‌شود. به عنوان مثال، در موتورهای جستجو، پسوندها حذف می‌شوند تا نتایج دقیق‌تر و سریع‌تری ارائه شود.
- **دسته‌بندی متن**: وقتی متنی به گروه‌های مختلف دسته‌بندی می‌شود، می‌توان از Stemming برای تبدیل کلمات به ریشه‌شان استفاده کرد.
- **پردازش زبان طبیعی (NLP)**: در پروژه‌های NLP که نیاز به پردازش زبان فارسی دارند، از Stemming برای تحلیل‌های معنایی، شناسایی موجودیت‌ها (NER) و تحلیل احساسات استفاده می‌شود.
- **جستجو و فیلترینگ متن**: در بسیاری از سیستم‌های جستجوی متنی و موتور جستجو، از Stemming برای کاهش کلمات به ریشه‌هایشان استفاده می‌شود تا جستجو سریع‌تر انجام شود و نتایج مرتبط‌تری ارائه شود.

مواردی که استفاده از Stemming بر Lemmatization ترجیح داده می‌شود:

- نیاز به **سرعت بالا** و **کارایی بیشتر** دارید.
- به **دقت معنایی کلمات** اهمیت نمی‌دهید.
- **حجم داده‌های زیاد** را باید پردازش کنید.
- در **جستجوهای متنی** و **سیستم‌های فیلتر کردن و دسته‌بندی** که فقط نیاز به شناسایی کلمات مشابه دارید، استفاده می‌شود.

Lemmatization معمولاً زمانی ترجیح داده می‌شود که **دقت معنایی** و **حفظ معنای دقیق کلمات** اهمیت داشته باشد.

2) Lemmatization:

Lemmatization فرآیندی است که برخلاف Stemming، که فقط به حذف پسوندها می‌پردازد، هدفش پیدا کردن **ریشه لغوی** و معنای دقیق کلمه با استفاده از اطلاعات گرامری و معنایی است. در این روش، با استفاده از دیکشنری‌ها و قوانین زبان‌شناسی، تغییرات دستوری کلمه مثل زمان فعل، تعداد، جنسیت و سایر ویژگی‌های گرامری بررسی می‌شود تا ریشه واقعی کلمه به دست آید. در صورتی که یک کلمه دارای چندین شکل گرامری باشد، Lemmatization سعی می‌کند که ریشه صحیح و معنادار آن را پیدا کند.

- **مزایا**:
  - دقت بالا در حفظ معنای کلمات
  - استفاده در کاربردهای پیچیده‌تری چون تحلیل‌های معنایی و ترجمه ماشینی
- **معایب**:
  - زمان‌بر و پیچیده‌تر از Stemming
  - نیاز به منابع دیکشنری و اطلاعات گرامری دقیق

مثال در زبان فارسی:

- کتاب → کتاب‌ها
- (ریشه فعل) دو → دویدن

برای درک بهتر در مثال انگلیسی:

- کلمه "running" به "run" تبدیل می‌شود.
- کلمه "better" به "good" تبدیل می‌شود (زیرا "better" درجه‌ای از "good" است)
- کلمه "happiness" به "happiness" باقی می‌ماند چون این کلمه خودش یک اسم اصلی است.

معروف‌ترین الگوریتم‌ها:

- WordNet Lemmatizer:
  - مبتنی بر پایگاه داده WordNet، توجه به زمان فعل، جمع، مفرد و جنسیت.
  - استفاده از دیکشنری معنایی و قواعد گرامری برای یافتن ریشه‌های دقیق.

- SpaCy Lemmatizer:

  - کتابخانه قدرتمند NLP با مدل‌های Machine Learning و Deep Learning.
  - دارای مدل‌های از پیش آموزش‌دیده برای دقت بالا و مناسب برای تحلیل احساسات و ترجمه ماشینی.

- Stanford Lemmatizer:

  - ابزار پردازش زبان طبیعی پیشرفته با تحلیل‌های گرامری دقیق.
  - مناسب برای کاربردهای پیچیده و زبان‌های مختلف به جز فارسی.

نسبت به زبان‌هایی مثل انگلیسی، ابزارهای Lemmatization برای فارسی هنوز به اندازه کافی توسعه نیافته‌اند. این الگوریتم‌ها نسبت به زبان‌های دیگر مثل انگلیسی پیچیده‌تر هستند، با این حال، چند ابزار وجود دارد:

- Hazm:

  - کتابخانه‌ای مشهور برای پردازش زبان فارسی.
  - علاوه بر Stemming، Lemmatization محدودی هم دارد.
  - به‌طور دقیق به معنی کلمات توجه نمی‌کند اما بهتر از روش‌های صرفاً Stemming عمل می‌کند.
  - Hazm Lemmatizer با استفاده از **دیکشنری‌های فارسی** و **قوانین دستوری**، کلمات فارسی را به ریشه معنایی دقیق نزدیک می‌کند.
  - این کتابخانه نیاز به یک **دیکشنری** دارد که معمولاً به صورت پیش‌فرض در آن گنجانده شده است.

- Parsivar:

  - کتابخانه‌ای برای پردازش زبان طبیعی فارسی.
  - شامل Lemmatization و **تحلیل صرفی** است.
  - دقیق‌تر از Hazm عمل می‌کند ولی هنوز محدودیت‌هایی دارد.

- FarsiStem:

  - FarsiStem یک کتابخانه دیگر برای پردازش متن فارسی است که شامل قابلیت‌های Lemmatization برای زبان فارسی می‌باشد.

کاربردها و استفاده‌ها:

- **پردازش زبان طبیعی (NLP)**: در تحلیل‌های معنایی و درک متن از Lemmatization استفاده می‌شود تا با حفظ معنی دقیق کلمات، آن‌ها به ریشه‌شان تبدیل شوند. برای مثال در سیستم‌های Sentiment Analysis، اگر کلمه "better" به "good" تبدیل شود، تحلیل‌های احساسی دقیق‌تری انجام خواهد شد.
- **ترجمه ماشینی و پردازش متنی**: وقتی که در برنامه‌های ترجمه ماشینی نیاز به تطبیق معنای دقیق کلمات باشد، Lemmatization مفید است.

بطور خلاصه:

- Stemming:

حذف **پسوندها** و تبدیل کلمه به ریشه به‌صورت سریع و الگوریتمی، بدون توجه به **معنای دقیق**. کاربرد:

- **موتورهای جستجو**: برای جستجوی سریع‌تر و مرتبط‌تر.
- **سیستم‌های فیلترینگ متن: برای دسته‌بندی متن‌ها و گروه‌بندی کلمات مشابه.**
- **تحلیل‌های اولیه داده‌ها که نیازی به دقت بالا در حفظ معنای دقیق کلمات ندارند.**

- Lemmatization:

یافتن ریشه معنایی دقیق کلمه با استفاده از قوانین گرامری و دیکشنری‌ها. کاربرد:

- **تحلیل‌های معنایی: برای درک دقیق متن و تحلیل احساسات.**
- **ترجمه ماشینی: برای ترجمه دقیق‌تر و درست‌تر.**
- **پردازش زبان طبیعی: در سیستم‌های چت‌بات و سیستم‌های تحلیل متن پیچیده.**

تفاوت‌های عملی:

در Stemming، یک کلمه می‌تواند به شکلی تغییر کند که لزوما با معنای اصلی‌اش تطابق نداشته باشد، دقت پایینی دارد اما سرعت بالاتر و پیچیدگی کمتری دارد به طور مثال کلمه "دویدن" در فرآیند Stemming به "دوید" تبدیل می‌شود، ولی این تغییر دقیقاً معنای ریشه‌ای فعل "دویدن" را نشان نمی‌دهد. اما در Lemmatization، هدف این است که فقط تغییرات لغوی حذف شوند تا **ریشه معنای درست کلمه** به دست آید. به این معنی که در حالت "دویدن"، ما "دو" را به عنوان ریشه درست می‌گیریم که به طور معنایی صحیح‌تر از "دوید" است.

برای مثال در فارسی برای Stemming در الگوریتم Hazm مراحل زیر طی می‌شوند:

- حذف جمع‌ها و پسوندهای جمع

  –ها، –های، –ان، –ین، –ات
- حذف پسوندهای ملکی

  –م، –ت، –ش، –مان، –تان، –شان
- حذف پسوندهای صفت ساز و قید ساز

  –ی، –ترین، –انه، –مند
- حذف پسوندهای فعلی

  –یم، –ید، –ند، –م، –ی، –د، –ه، –تر، –ترین
- حذف نشانه‌های اضافی

  حذف **کسره اضافه** یا **ی نکره** در انتهای کلمات.

جدول نمونه‌ها: (در نظر گرفته‌ایم که Stemming فقط حذف پسوند دارد.)

کلمه ورودی	Stemming	Lemmatization
دویدند	دوید	دویدن
برمی‌گردانند	برمی‌گردان	برگردان
کتابهایشان	کتاب	کتاب
نشسته‌اند	نشست	نشستن
بافته‌ایم	بافت	بافتن
آمده‌اید	آمد	آمدن
بهترین	به	خوب
می‌رود	می‌رو	رفتن

این موارد پیش‌بینی ما هستند، اما یک نمونه پیاده‌سازی هم برای آن در فایل Q1.ipynb انجام شده‌است.

۳. سوال شماره ۳:

سه کلمه و سه بردار جانمایی داده شده است، به نظر شما کدام بردار مربوط به کلمه apple است؟ چرا؟

King, Queen, Apple

1) **[0.1, -0.9, 0.2]**
2) **[0.7, 0.5, 0.3]**
3) **[0.8, 0.6, -0.4]**

در اینجا در واقع باید ارتباط بین کلمات و بردارهای عددی را در نظر بگیریم. (Word Embeddings)

توضیح تئوری در مورد Word Embedding:

Word Embedding یعنی تبدیل کلمات به بردارهای عددی در یک فضای چندبعدی، به‌طوری‌که کلماتی که از نظر مفهومی به هم نزدیک هستند، بردارهای عددی مشابهی داشته باشند. به این بردارها Dense Vectors گفته می‌شود و برخلاف روش‌های قدیمی مثل Bag of Words یا TF-IDF که بردارهای پراکنده (Sparse Vectors) بودند، این‌ها کاملاً فشرده و کارآمدند. از مزایای این روش می‌توان به دقت بالا، کارایی بالا و پردازش سریعتر و همچنین قابلیت یادگیری آن اشاره کرد. بطور کلی کاربردهای word embedding نیز به شرح زیر است:

- **Semantic Analysis**: (Semantic Matching, Semantic Search, Relationship Analysis (Word Similarity, Analogy Detection, and Knowledge Graphs))

- **Text Classification**: (Sentiment Analysis, Topic Modeling, Anomaly Detection, NER, Intent Detection (which is used heavily in Chatbots))

- **Information Retrieval**: (Search Engines, Question Answering Systems, NER (it helps extract relevant entities for queries))

- **NLG (Natural Language Generation)**: (text generation, summarization, response generation (for Chatbots))

- **Spelling Correction**

حل سوال:

کلماتی که از لحاظ معنایی به هم شبیه‌اند، در این فضای چندبُعدی به هم نزدیک‌اند، برای همین اگر فاصله بردارهایی که داده شده را با هم محاسبه کنیم، ۲ برداری که فاصله کتری با یکدیگر داشته باشند، با احتمال زیادی کلمات King و Queen هستند، و برداری که از بقیه فاصله بیشتری داشته باشد، کلمه Apple خواهد بود. برای محاسبه فاصله، از **فاصله اقلیدسی** (Euclidean Distance) استفاده می‌کنیم:

$$A = [a_1. a_2. a_3]. B = [b_1. b_1. b_1]$$

$$d = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$$

بردارهای صورت سوال:

1) [0.1, -0.9, 0.2]
2) [0.7, 0.5, 0.3]
3) [0.8, 0.6, -0.4]

$$d_{1.2} = \sqrt{(0.1 - 0.7)^2 + (-0.9 - 0.5)^2 + (0.2 - 0.3)^2} = \sqrt{0.36 + 1.96 + 0.01} = \sqrt{2.33} \cong 1.5264$$

$$d_{1.3} = \sqrt{(0.1 - 0.8)^2 + (-0.9 - 0.6)^2 + (0.2 + 0.4)^2} = \sqrt{0.49 + 2.25 + 0.36} = \sqrt{3.1} \cong 1.7607$$

$$d_{2.3} = \sqrt{(0.7 - 0.8)^2 + (0.5 - 0.6)^2 + (0.3 + 0.4)^2} = \sqrt{0.01 + 0.01 + 0.49} = \sqrt{0.51} \cong 0.7141$$

نتیجه‌گیری:

نتایج بالا نشان می‌دهند که فاصله بردار ۲ و ۳ کم است، این یعنی به احتمال زیاد این ۲ بردار باید مربوط به کلمات King و Queen باشند، به همین تناسب فاصله بین بردار ۱ و ۲، و همچنین ۱ و ۳، بیشتر است که روی فرضیه جمله قبل صحه می‌گذارد.

۴. سوال شماره ۴:

با استفاده از ۵ جمله زیر:

<s> the quick brown fox jumps over the lazy dog </s>
<s> the lazy dog barks at the quick fox </s>
<s> a quick brown fox jumps over the lazy dog </s>
<s> the lazy dog sleeps all day </s>
<s> the quick fox jumps over the lazy dog </s>

احتمال جمله زیر را با توجه به bigramهای آن محاسبه کنید، از add-one smoothing استفاده کنید.

<s> the lazy fox jumps </s>

توضیح تئوری در مورد bigrams:

مدل زبانی bigram، یکی از مدل‌های ساده برای **محاسبه احتمال جملات** است که به جای در نظر گرفتن کلمات به صورت مستقل، به وابستگی بین هر دو کلمه متوالی توجه می‌کند، به عبارتی یک bigram دنباله‌ای از **دو کلمه‌ی متوالی** در یک متن است. بنابراین bigramها به ما کمک می‌کنند که **وابستگی بین دو کلمه‌ی پشت سر هم** را بررسی کرده و احتمال وقوع یک کلمه را با توجه به کلمه‌ی قبلی تخمین بزنیم. احتمال یک جمله نیز به عنوان حاصل ضرب احتمالات دوتایی‌های پشت سر هم (Bigram) محاسبه می‌شود. کاربردها:

Language Models: bigramها به عنوان بخشی از مدل‌های زبانی برای محاسبه احتمال وقوع یک کلمه بر اساس کلمه قبلی استفاده می‌شوند. برای تولید متن (Text Generation)، تکمیل خودکار (Autocomplete) و سیستم‌های پیشنهاد دهنده (Recommendation Systems) استفاده می‌شوند.

Speech Recognition: bigramها به کمک مدل‌های زبانی برای تشخیص دقیق‌تر کلمات از صوت استفاده می‌شوند و باعث بهبود دقت تشخیص گفتار با مدل‌سازی توالی‌های محتمل می‌شوند. در تشخیص گفتار پزشکی و حقوقی کاربرد دارد، همچنین در Siri، Google Assistant، Alexa بکار می‌روند.

Phrase Search: از bigramها برای پیدا کردن عبارات خاص و پرتکرار در متون بزرگ با استفاده از Inverted Index استفاده می‌شود. همینطور در موتورهای جستجو مانند Google, Bing و همچنین در سیستم‌های جستجوی داده مانند ElasticSearch کاربرد دارد.

Spell Correction: از bigramها برای تصحیح غلط‌های املایی یا تایپی با بررسی کلمات پرتکرار پس از یک کلمه داده شده، استفاده می‌شود. در ویرایش‌گرهای متن مثل Microsoft Word, Google Docs و همچنین در موتورهای جستجو مانند Google Suggestion کاربرد دارد.

Language Identification: bigramها به کمک شناسایی الگوهای خاص برای تشخیص زبان متن استفاده می‌شوند. و در مترجم‌ها مانند Google Translate و همچنین در تحلیل داده‌های شبکه‌های اجتماعی کاربرد دارند.

Sentiment Analysis: الگوهای زبانی مرتبط با احساسات مثبت یا منفی از طریق bigramها تشخیص داده می‌شوند. و در بررسی نظرات مشتریان و یا تحلیل داده‌های شبکه‌های اجتماعی کاربرد دارند.

Text Analysis: از bigramها برای شناسایی عبارات پرتکرار و استخراج اصطلاحات خاص استفاده می‌شود. در شناسایی اصطلاحات علمی یا حقوقی پرتکرار و یا پیدا کردن موضوعات کلیدی در متن‌های بزرگ کاربرد دارد. (یافتن ترکیب‌های پرتکرار (مثلاً "سلام علیکم" در متون فارسی))

Plagiarism Detection: با مقایسه bigramهای متون برای شناسایی شباهت‌های غیرعادی و یافتن سرقت علمی یا کپی‌برداری استفاده می‌کنیم. در بررسی مقاله‌های علمی جهت تشخیص تقلب و همچنین شناسایی کپی‌برداری در محتوای تولید شده نیز کاربرد دارند.

Text Mining: کشف الگوها، اصطلاحات رایج، و روابط بین کلمات در مجموعه‌های بزرگ داده با استفاده از bigramها انجام می‌شود. در استخراج اطلاعات از مقالات علمی و یا تحلیل متون قانونی یا پزشکی نیز کاربرد دارد.

Data Compression: برای کاهش حجم داده‌های متنی با شناسایی الگوهای تکراری از bigramها ستفاده می‌شود. در الگوریتم‌های فشرده‌سازی متنی مانند Huffman Coding کاربرد دارد.

Recommender Systems: از bigramها برای پیشنهاد عبارت‌های بعدی یا تکمیل خودکار متن استفاده می‌شود. در موتورهای جستجو و یا پیشنهادات مبتنی بر متن (مثل نوشتن ایمیل‌ها) کاربرد دارند.

NLP and chatbots systems: از bigramها برای تولید پاسخ‌های منطقی و تحلیل بهتر قصد کاربر استفاده می‌شود. در چت‌بات‌های تعاملی مانند ChatGPT, Siri, Alexa و ربات‌های پشتیبانی مشتری کاربرد دارند.

همچنین قابل ذکر است که Bigram یک حالتی از K-gramهاست که به یک توالی از k کلمه پشت سر هم گفته می‌شود. (وقتی K را افزایش بدهیم، مدل دقیق‌تر ولی پیچیده‌تر و حساس‌تر به داده‌های آموزشی می‌شود. – روش‌های هیوریستیک نیز در مواقعی که داده‌ی آموزشی محدود است یا نمی‌خواهیم آزمون و خطای زیادی انجام دهیم، مفید هستند.)

در نهایت روی bigramها یک Inverted Index می‌زنیم، تا سریعاً ببینیم هر bigram در کجاها (کدام جمله‌ها یا اسناد) ظاهر شده است. این تکنیک در بازیابی اطلاعات (Information Retrieval) و پردازش زبان طبیعی (NLP) کاربرد گسترده‌ای دارد. یک Inverted Index نوعی ساختار داده است که به جای ذخیره کردن داده‌ها به صورت ترتیبی (Straight Index)، محل وقوع داده‌ها را ذخیره می‌کند.

برای اینکار ابتدا از هر جمله همه‌ی bigramها رو استخراج می‌کنیم، سپس از یک دیکشنری (Python Dictionary) **یا** HashMap **برای ایجاد ساختار داده‌ای** استفاده می‌کنیم، نهایتا برای هر بیگرام، لیستی از ID های جملاتی که آن بیگرام را دارند، ذخیره می‌کنیم.

- در جستجوی متنی: کلمات (Terms) به عنوان کلید (Key) استفاده می‌شوند و لیستی از موقعیت‌های آن کلمات به عنوان مقدار (Value) ذخیره می‌شود.
- در k-gramها (مثل bigram): توالی‌های دو کلمه‌ای به عنوان کلیدها استفاده می‌شوند.

حل سوال:

سوال از ما می‌خواهد که **احتمال جمله‌ی جدید** <s> the lazy fox jumps </s> را با استفاده از داده‌های موجود (پنج جمله داده شده) محاسبه کنیم. اما چون ممکن است بعضی از bigramهای جمله‌ی جدید در داده‌های ما نباشند، باید از تکنیکی به نام **Add-One Smoothing** یا **Laplace Smoothing** استفاده کنیم.

به صورت پیش‌فرض، احتمال یک کلمه با توجه به کلمه قبلی (احتمال bigram) به صورت زیر محاسبه می شود:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Symbol	Meaning
$P(w_i \mid w_{i-1})$	Probability of word w_i given the previous word w_{i-1}
$C(w_{i-1}, w_i)$	Count of the bigram (w_{i-1}, w_i)
$C(w_{i-1})$	Count of the word w_{i-1} (i.e., how often it appears)

اما اگه یک bigram هرگز در داده‌ها نباشد (یعنی تعداد وقوعش صفر باشد)، در حالت معمول احتمالش هم صفر می‌شود که مشکل‌ساز هست. Add-One Smoothing این مشکل را با اضافه کردن ۱ به تمام شمارش‌ها حل می‌کند:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

Symbol	Meaning
$C(w_{i-1}, w_i)$	Count of the bigram (w_{i-1}, w_i)
$C(w_{i-1})$	Count of the word w_{i-1} (total occurrences of the first word in the corpus)
V	Vocabulary size (total number of unique words)
$+1$	The smoothing factor (adds 1 to every possible bigram)

بنابراین ما برای جمله‌ی جدید احتمال هر bigram را با استفاده از Add-One Smoothing محاسبه می‌کنیم و در نهایت همه‌ی آنها را در هم ضرب می‌کنیم تا احتمال کلی آن بدست‌آید.

نکته مهم: در این سوال از آنجایی که نشانه‌های شروع و پایان وجود دارد (<s> و </s>)، اینها هم بخشی از bigramها به حساب می‌آیند. این ها اغلب در مدل سازی زبان برای نشان دادن آغاز و پایان یک جمله استفاده می شوند.

ابتدا Vocabulary Size را برای جملات مثال بدست می‌آوریم، سپس جدول bigramها را در یک جدول کلی محاسبه می‌کنیم (در واقع تک تک جملات حساب شده‌اند و مجموعه آنها در یک جدول قرار می‌گیرد):

<s> the quick brown fox jumps over the lazy dog </s>
<s> the lazy dog barks at the quick fox </s>
<s> a quick brown fox jumps over the lazy dog </s>
<s> the lazy dog sleeps all day </s>
<s> the quick fox jumps over the lazy dog </s>

جمله مورد نظر:

<s> the lazy fox jumps </s>

1) All Unique Words:

<s>, the, quick, brown, fox, jumps, over, lazy, dog, barks, at, a, sleeps, all, day, </s>

Vocabulary Size (V_global) = 16

2) Total Unique bigrams Count (combined all)

Bigram	Total Count
<s> the	4
the quick	3
quick brown	2
brown fox	2
fox jumps	3
jumps over	3
over the	3
the lazy	5
lazy dog	5
dog </s>	3
dog barks	1
barks at	1
at the	1
quick fox	2
fox </s>	1
<s> a	1
a quick	1
dog sleeps	1
sleeps all	1
all day	1
day </s>	1

3) Total Unique Unigrams Count

Unigram	Count
<s>	5
the	8
quick	4
brown	2
fox	4
jumps	3
over	3
lazy	5
dog	5
barks	1
at	1
a	1
sleeps	1
all	1
day	1
</s>	5

4) Bigrams for Given sentence from previous table:

Bigram	Count
<s> the	4
the lazy	5
lazy fox	0
fox jumps	3
jumps </s>	0

5) Unigrams for Given sentence from previous table:

Bigram	Count
<s>	5
the	4
lazy	5
fox	0
jumps	3
</s>	0

6) Probability Calculation for Each Bigram



We used Add-One Smoothing formula. (with V_global = 16)

1. <s> the

$$P(\text{the} \mid \text{<s>}) = \frac{C(\text{<s> the}) + 1}{C(\text{<s>}) + V} = \frac{4+1}{5+16} = \frac{5}{21} \approx 0.2381$$

2. the lazy

$$P(\textit{lazy} \mid \textit{the}) = \frac{C(\textit{the lazy}) + 1}{C(\textit{the}) + V} = \frac{5+1}{8+16} = \frac{6}{24} = \frac{1}{4} = 0.2500$$

3. lazy fox

$$P(\text{fox} \mid \text{lazy}) = \frac{C(\text{lazy fox}) + 1}{C(\text{lazy}) + V} = \frac{0+1}{5+16} = \frac{1}{21} = 0.0476$$

4. fox jumps

$$P(\textit{jumps} \mid \textit{fox}) = \frac{C(\textit{fox jumps}) + 1}{C(\textit{fox}) + V} = \frac{3+1}{4+16} = \frac{4}{20} = \frac{1}{5} = 0.2000$$

5. jumps </s>

$$P(\text{</s>} \mid \textit{jumps}) = \frac{C(\textit{jumps </s>}) + 1}{C(\textit{jumps}) + V} = \frac{0+1}{3+16} = \frac{1}{19} = 0.0526$$

7) Multiply All Probabilities Together

$$P(\text{sentence}) = P(\text{the} \mid \text{<s>}) \times P(\text{lazy} \mid \text{the}) \times P(\text{fox} \mid \text{lazy}) \times P(\text{jumps} \mid \text{fox}) \times P(\text{"</s>"} \mid \text{"jumps"})$$

$$\text{Total Probability} = 0.2381 \times 0.25 \times 0.0476 \times 0.2 \times 0.0526 = 0.0000298072628 \approx 3 * 10^{-5}$$

در نهایت احتمال وجود این جمله $3 * 10^{-5}$ می‌باشد.

۵. سوال شماره ۵:

در نوتبوک Q5.ipynb با استفاده از نامپای، جانمایی word2vec را از پایه پیاده سازی کنید.

Word2Vec is a popular technique for word embeddings, which captures the meaning of words by placing them in a continuous vector space. In this exercise, you will implement Word2Vec using NumPy and complete the missing parts of the code. We will represent each word as a one-hot vector, meaning each word in the vocabulary is mapped to a unique binary vector with only one active (1) position.

توضیحات تئوری:

Word Embedding روشی برای تبدیل کلمات به بردار های عددی می‌باشد، به‌گونه‌ای که **روابط معنایی و نحوی بین کلمات** در این نمایش‌های عددی حفظ شود. این فرآیند به مدل‌های یادگیری ماشین و شبکه‌های عصبی کمک می‌کند تا مفاهیم معنایی کلمات را بهتر درک و پردازش کنند. این روش شامل تکنیک‌هایی مانند **Word2Vec، GloVe و FastText** است.

Word2Vec تکنیکی است که توسط مهندسان گوگل در سال ۲۰۱۳ معرفی شد و با جملاتی مانند «پادشاه - مرد + زن = ملکه» محبوبیت یافت.

این مدل یادگیری ماشین به دلیل سادگی، دقت بالا و توانایی درک روابط معنایی بین کلمات، به یکی از محبوب‌ترین روش‌های Word Embedding تبدیل شده است. این مدل از شبکه‌های عصبی ۲ لایه کم‌عمق استفاده می‌کند و کلمات با معنای مشابه را به بردارهای نزدیک به هم تبدیل می‌کند. ابعاد این بردارها برای تمامی کلمات یک متن ثابت است و هدف اصلی مدل، ایجاد نمایش‌های عددی قابل فهم و مؤثر از کلمات است.

این مدل به صورت خاص از داده‌های بدون برچسب (متون خام) برای آموزش استفاده می‌کند اما اگرچه Word2Vec به عنوان یک مدل یادگیری بدون نظارت (Unsupervised Learning) شناخته می‌شود، فرآیند یادگیری آن به دلیل استفاده از مکانیزم پیش‌بینی هدف (با استفاده از تابع هزینه)، شباهت زیادی به یادگیری نظارت‌شده (supervised Learning) دارد.

با این حال، برچسب‌ها (Targets) به‌طور خودکار از داده‌ها استخراج می‌شوند و توسط انسان تعیین نمی‌شوند؛ به همین دلیل Word2Vec همچنان به عنوان یک مدل بدون نظارت محسوب می‌شود.

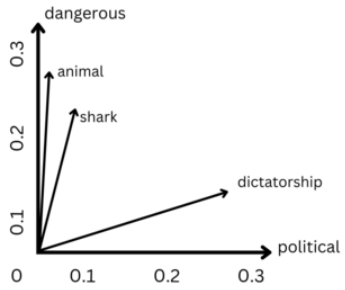
استفاده از دو مدل CBOW و Skip-gram امکان تنظیم بهتر مدل بر اساس حجم داده‌ها و نوع کاربرد را فراهم می‌کند. این روش به‌ویژه در تحلیل زبان طبیعی (NLP) و کاربردهای مبتنی بر متن بسیار مؤثر است.

۱. CBOW (Continuous Bag of Words): در این مدل، کلمات زمینه (Context Words) به عنوان داده ورودی و کلمه مرکزی (Target Word) به عنوان خروجی پیش‌بینی می‌شود. این رویکرد در متن‌هایی با حجم بالا کارآمدتر است.
۲. Skip-gram: در این مدل، کلمه مرکزی به عنوان داده ورودی و کلمات زمینه به عنوان خروجی‌های هدف در نظر گرفته می‌شوند. این روش برای تشخیص روابط معنایی نادر و کلمات کمتر استفاده‌شده مناسب‌تر است.

نکته: Word2Vec با استفاده از یک پنجره کلمه (Window Size) کلمات نزدیک به هم را یاد می‌گیرد. بنابراین به طور غیرمستقیم ارتباطات شبیه به Bigram و Trigram را پوشش می‌دهد. (FastText نسخه بهبود یافته‌ی Word2Vec از زیرکلمات (subwords) استفاده می‌کند که شباهت بیشتری به‌N-gram ها دارد.)

نظریه های تئوری مرتبط:

فرضیه توزیعی (Distributional Hypothesis) بیان می‌کند که کلماتی که در زمینه‌های مشابه ظاهر می‌شوند، معمولاً معنای مشابهی دارند (*J.R. Firth (1957)* –). Word2Vec و سایر تکنیک‌های Word Embedding بر پایه این فرضیه بنا شده‌اند و با تحلیل هم‌نشینی کلمات در متون، بردارهای عددی‌ای ایجاد می‌کنند که مفاهیم مشابه را به‌هم نزدیک می‌کند. این روش به‌جای تعریف کلمات بر اساس لغت‌نامه، از کاربرد آن‌ها در متن استفاده می‌کند و در پردازش زبان طبیعی (NLP) برای وظایفی مانند طبقه‌بندی متن، ترجمه، تحلیل احساسات و بازیابی اطلاعات مؤثر است. مثال:



توضیحات پیاده‌سازی:

در اینجا یک مدل Word2Vec با معماری CBOW (Continuous Bag-of-Words) پیاده‌سازی کرده‌ایم. به‌طور کلی، یک کلاس برای پیاده‌سازی توابع مختلف و انجام مراحل مدل‌سازی ایجاد شده است.

مراحل کلی عملکرد مدل:

۱) بارگزاری کتابخانه‌ها:

ابتدا کتابخانه‌های مرتبط را بارگزاری می‌کنیم و سپس به پیاده‌سازی مراحل می‌پردازیم.

```
import necessary libraries

import numpy as np
from collections import defaultdict
✓ 0.1s
```

NumPy یک کتابخانه‌ی متن‌باز برای محاسبات عددی و علمی است که با آرایه‌ها و ماتریس‌ها کار کرده و عملیات ریاضی را با سرعت بالا انجام می‌دهد.

defaultdict از ماژول collections پایتون، نوعی دیکشنری است که به‌جای ایجاد خطای KeyError برای کلیدهای ناموجود، مقدار پیش‌فرض تعیین‌شده‌ای را برمی‌گرداند.

۲) مقداردهی اولیه (Initialization)

- تعریف و تنظیم هایپرامترها شامل:
  - تعداد دفعات آموزش (epochs)
  - نرخ یادگیری (learning_rate)
  - اندازه‌ی بردار تعبیه‌ای (embed_size)
  - اندازه‌ی پنجره‌ی بافت (window_size)
  - اندازه‌ی هر دسته‌ی آموزشی (batch_size)

۳) ساخت واژگان (Vocabulary Creation)

این تابع مسئول ساخت واژگان (Vocabulary) از یک مجموعه داده‌ی متنی (corpus) است. به بیان ساده، این تابع تمام کلمات موجود در متن را استخراج کرده و به هر کلمه **یک** اندیس منحصربه‌فرد تخصیص می‌دهد. همچنین هنگام تولید داده‌های آموزشی صدا زده می‌شود.

```
def get_vocab(self, corpus):

    word_counts = defaultdict(int)

    for word in corpus:
        word_counts[word] += 1

    vocab_size = len(word_counts)
    words_list = list(word_counts.keys())

    word2idx = {word: i for i, word in enumerate(words_list)}
    idx2word = {i: word for i, word in enumerate(words_list)}

    return words_list, vocab_size, word2idx, idx2word
```

- ابتدا تمام کلمات منحصربه‌فرد از متن ورودی (corpus) را استخراج می‌کند.
- سپس از طریق دیکشنری ساخته شده از کلمات منحصربه‌فرد، دو دیکشنری برای نگاشت کلمات به اندیس‌ها، و برعکس می‌سازد:
  - word_index: تبدیل کلمه به عدد.
  - index_word: تبدیل عدد به کلمه.

۴) تولید داده‌های آموزشی (Generating Training Data)

این تابع مسئول تولید داده‌های آموزشی برای مدل Word2Vec با استفاده از معماری CBOW (Continuous Bag-of-Words) است. این تابع داده‌های آموزشی را به‌صورت زوج‌هایی از کلمه هدف و کلمات بافت (Context Words) آماده می‌کند.

```python
def generate_training_data(self, corpus):

    # part 1
    self.words_list, self.v_count, self.word_index, self.index_word = self.get_vocab(corpus)

    # part 2
    X = []
    y = []
    for index, word in enumerate(corpus):
        prev = self.prev_words(index, corpus)
        next = self.next_words(index, corpus)
        assert len(prev) == len(next)
        X.append(prev + next)
        y.append([self.word_index[word]])

    X = np.array(X)
    y = np.array(y).flatten()

    training_data = list(zip(X, y))

    return training_data
```

- ابتدا لیست کلمات، تعداد کلمات منحصربه‌فرد و دیکشنری‌های word_index و index_word را از تابع get_vocab دریافت می‌کند.
- سپس برای هر کلمه در متن، کلمات اطراف آن را در محدوده‌ی تعیین‌شده توسط (window_size) به‌عنوان بافت یا Context انتخاب می‌کند. این کار با استفاده از دو تابع prev_words و next_words انجام می‌شود.

```python
# Function to get previous words in the window
def prev_words(self, i, doc):
    out = []
    window = self.window_size//2
    for index in range(i - window, i):
        if index >= 0:
            out.append(self.word_index[doc[index]])
        else:
            out.append(0)  # 0 = <PAD>
    return out

# Function to get next words in the window
def next_words(self, i, doc):
    out = []
    window = self.window_size//2
    for index in range(i + 1, i + window + 1, 1):
        if index < len(doc):
            out.append(self.word_index[doc[index]])
        else:
            out.append(0)  # 0 = <PAD>
    return out
```

- prev_words: این تابع کلمات **قبلی** از موقعیت مشخص‌شده (i) را جمع‌آوری می‌کند.
- next_words: این تابع کلمات **بعدی** از موقعیت مشخص‌شده (i) را جمع‌آوری می‌کند.

برای مثال اگر پنجره ۵ باشد، فقط نصف آن یعنی ۲ کلمه قبل و ۲ کلمه بعد انتخاب می‌شوند. همچنین اگر ایندکس کلمه انتخابی خارج از بازه کلمات باشند، به‌جای آن صفر اضافه می‌کند. (0 نشان‌دهنده‌ی <PAD> است.)

در نهایت داده‌های آموزشی شامل زوج‌هایی از کلمه‌ی هدف و کلمات بافت ساخته و برگردانده می‌شوند.

۵) آموزش مدل (Training)

این تابع وظیفه دارد مدل Word2Vec را با استفاده از داده‌های آموزشی داده شده، آموزش دهد. از روش CBOW استفاده می‌کند و از طریق انتشار پیشرو (Forward Propagation) و انتشار معکوس (Backpropagation) وزن‌ها را بهینه‌سازی می‌کند.

```
def train(self, training_data):

    # Initialize weights randomly
    self.w1 = np.random.rand(self.v_count, self.embed_size)
    self.w2 = np.random.rand(self.embed_size, self.v_count)

    for epoch in range(self.epochs):
        total_loss = 0

        # Initialize gradients and batch counter
        batch_grad_w1 = np.zeros_like(self.w1)
        batch_grad_w2 = np.zeros_like(self.w2)
        batch_count = 0

        for i, (x, target) in enumerate(training_data):

            # Generate one-hot encoded vectors for context words (x)
            context_vectors = [self.word2onehot(self.index_word[word_index]) for word_index in x]

            # Compute the average of context vectors to form the input vector (CBOW style)
            input_vector = np.mean(context_vectors, axis=0)

            # Generate one-hot encoded vector for the target word
            y_true = self.word2onehot(self.index_word[target])

            # Forward pass
            y_predicted, hidden_layer_output, _ = self.forward_pass(input_vector)
```

```
            # Call the backpropagation function for training
            loss, batch_grad_w1, batch_grad_w2, batch_count = self.backpropagation(
                input_vector, y_true, hidden_layer_output, y_predicted,
                batch_grad_w1, batch_grad_w2, batch_count,
                training_complete=(i + 1 == len(training_data))
            )

            # Accumulate total loss
            total_loss += loss
```

- ابتدا دو ماتریس وزن به‌صورت تصادفی مقداردهی می‌شوند:
  - w1: وزن بین لایه ورودی و لایه پنهان (Embedding Vectors) - (ابعاد: (v_count, embed_size))
  - w2: وزن بین لایه پنهان و لایه خروجی (ابعاد: (embed_size, v_count))
- در ادامه مدل به تعداد self.epochs آموزش داده می‌شود.
- سپس متغیرهای مربوط به گرادیان و خطا را تعریف می‌کنیم:
  - Loss: مجموع خطا برای محاسبه میانگین خطا در انتهای هر دوره.
  - batch_grad_w1 و batch_grad_w2: برای جمع‌آوری گرادیان‌های محاسبه‌شده در هر دسته (Batch).
  - batch_count: تعداد داده‌هایی که در هر دسته پردازش شده‌اند.
- **به ازای هر نمونه آموزشی مراحل بعدی طی خواهند شد.** (X: مجموعه‌ای از ایندکس‌های کلمات Context، Target: ایندکس کلمه هدف)
- تولید بردار ورودی (Input_Vector):
  - ابتدا هر کلمه از x به صورت One-Hot Encoding تبدیل می‌شود.
  - سپس با میانگین‌گیری از همه بردارهای بافت، یک بردار ورودی (Input Vector) تولید می‌شود. (این بخش مربوط به معماری CBOW است.) (در واقع دارد **عملکردی مشابه با GlobalAveragePooling1D** را انجام می‌دهد.)
- تولید بردار هدف (y_true): کلمه هدف را به بردار One-Hot Encoding تبدیل می‌کند.
- انجام انتشار پیشرو (Forward Propagation):

در این تابع خروجی پیش‌بینی‌شده بر اساس ورودی داده شده **و** وزن‌های مدل تولید می‌شود.

```
def softmax(self, x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum(axis=0)


def forward_pass(self, input_vector):
    hidden_layer_output = np.dot(self.w1.T, input_vector)
    output_scores = np.dot(self.w2.T, hidden_layer_output)
    predicted_probabilities = self.softmax(output_scores)
    return predicted_probabilities, hidden_layer_output, output_scores
```

  - با محاسبه ضرب داخلی بردار ورودی با ماتریس وزن اول (w1) خروجی لایه پنهان بدست می‌آید، hidden_layer_output یک بردار با طول embed_size است که نشان‌دهنده‌ی Embedding Vector است.
  - با محاسبه ضرب داخلی hidden_layer_output با ماتریس وزن دوم (w2) امتیازات خروجی بدست می‌آید، output_scores یک بردار با طول v_count است که نشان‌دهنده‌ی امتیازهای خام برای هر کلمه‌ی موجود در واژگان است.
  - output_scores از مرحله قبل به تابع softmax داده می‌شود، و آن مقادیر را به احتمالاتی بین 0 و 1 تبدیل می‌کند که مجموع آن‌ها برابر با 1 است، predicted_probabilities برداری با طول v_count است که احتمال هر کلمه از واژگان را نشان می‌دهد.

- انجام انتشار معکوس (Backpropagation):

در این تابع فرآیند Backpropagation به صورت یکپارچه اجرا می‌شود، این شامل محاسبه‌ی خطا، محاسبه‌ی گرادیان‌ها، به‌روزرسانی وزن‌ها و بازنشانی گرادیان‌ها برای هر Batch است. (از روش BGD استفاده کرده‌ایم.)

- محاسبه خطا: ابتدا خطا یا Error به عنوان تفاوت بین پیش‌بینی مدل (y_predicted) و مقدار واقعی (y_true) محاسبه می‌شود. y_true و y_predicted هر دو بردارهایی با طول برابر با تعداد کلمات واژگان هستند. تابع خطا با Categorical Cross-Entropy Loss محاسبه می‌شود. اضافه کردن مقدار 1e-9 برای جلوگیری از خطای لگاریتم صفر است.
- محاسبه گرادیان‌ها: در ادامه برای محاسبه گرادیان‌ها، مشتقات جزئی برای وزن‌های w1 و w2 محاسبه می‌شوند تا اثر تغییرات در این وزن ها را نشان دهند با تابع np.outer() ضرب خارجی دو بردار را محاسبه شده و یک ماتریس تولید می‌شود.
- جمع‌آوری گرادیان‌ها: گرادیان‌ها به متغیرهای batch_grad_w1 و batch_grad_w2 اضافه می‌شوند، در واقع به جای به‌روزرسانی وزن‌ها بلافاصله پس از هر نمونه، گرادیان‌ها برای چند نمونه جمع‌آوری می‌شوند، این روش Batch Gradient Descent است و باعث پایداری بیشتر آموزش مدل می‌شود.
- به‌روزرسانی وزن‌ها (Weights Update): اگر تعداد داده‌های پردازش‌شده به اندازه‌ی batch_size برسد یا داده‌ها به پایان برسند، آنگاه وزن‌ها به‌روزرسانی می‌شوند.
- میانگین‌گیری گرادیان‌ها (Gradient Averaging): از آنجایی که از BGD استفاده می‌کنیم، باید میانگین گرادیان‌ها محاسبه شود، هر گرادیان بر تعداد نمونه‌های پردازش‌شده (batch_count) تقسیم می‌شود.
- به‌روزرسانی وزن‌ها با استفاده از Gradient Descent : وزن‌ها با استفاده از فرمول Gradient Descent به‌روزرسانی می‌شوند
- بازنشانی گرادیان‌ها و شمارنده: پس از به‌روزرسانی وزن‌ها، متغیرهای batch_grad_w1, batch_grad_w2 و batch_count بروزرسانی می‌شوند تا برای Batch جدید آماده باشند.

```
def backpropagation(self, input_vector, y_true, hidden_layer_output, y_predicted,
                    batch_grad_w1, batch_grad_w2, batch_count, training_complete=False):

    # Step 1: Calculate the error (difference between predicted and true labels)
    error = y_predicted - y_true
    loss = -np.log(np.dot(y_true, y_predicted) + 1e-9)

    # Step 2: Calculate gradients using backpropagation
    gradient_w2 = np.outer(hidden_layer_output, error)
    gradient_w1 = np.outer(input_vector, np.dot(self.w2, error))

    # Step 3: Accumulate gradients for Batch Processing
    batch_grad_w1 += gradient_w1
    batch_grad_w2 += gradient_w2
    batch_count += 1

    # Step 4: Update weights if batch is completed or training is finished
    if training_complete or (batch_count == self.batch_size):
        # Average the gradients for batch processing
        batch_grad_w1 /= batch_count
        batch_grad_w2 /= batch_count

        # Update weights using Gradient Descent
        self.w1 -= self.lr * batch_grad_w1
        self.w2 -= self.lr * batch_grad_w2

        # Reset gradients and batch count for the next batch
        batch_grad_w1 = np.zeros_like(self.w1)
        batch_grad_w2 = np.zeros_like(self.w2)
        batch_count = 0

    return loss, batch_grad_w1, batch_grad_w2, batch_count
```

- استخراج Embedding Vectors و محاسبه‌ی شباهت معنایی بین کلمات:

در این ۲ تابع کلمات مشابه با یک کلمه‌ی مشخص انتخاب می‌شوند. این قابلیت بسیار مهمی است که نشان می‌دهد مدل معنای کلمات را به‌درستی یاد گرفته است.

```
def word_vec(self, word):
    word_index = self.word_index[word]
    word_embedding = self.w1[word_index]
    return word_embedding

def vec_sim(self, word, top_n):
    v_w1 = self.word_vec(word)
    similarity = {}

    for other_word in self.words_list:
        if other_word == word:
            continue
        v_w2 = self.word_vec(other_word)
        sim = np.dot(v_w1, v_w2) / (np.linalg.norm(v_w1) * np.linalg.norm(v_w2))
        similarity[other_word] = sim

    words_sorted = sorted(similarity.items(), key=lambda x: x[1], reverse=True)

    for word, sim in words_sorted[:top_n]:
        print(f'{word}: {sim}')
```

- **تابع** word_vec(): این تابع برای یک کلمه‌ی خاص، Embedding Vector آن را استخراج می‌کند. این بردارها از ماتریس w1 استخراج می‌شوند که طی فرآیند آموزش یاد گرفته شده‌اند.
- تابع vec_sim(): این تابع کلمات مشابه با یک کلمه‌ی داده‌شده را پیدا می‌کند و آن‌ها را به ترتیب شباهت نمایش می‌دهد. شباهت بین کلمات از طریق ضرب داخلی (Dot Product) و Cosine Similarity محاسبه می‌شود.

پیشنهادات برای بهبود مدل:

✓ بهبود دقت مدل (Quality Improvement)

- افزایش حجم داده‌ها (مثل Wikipedia به جای Text8)
- افزایش ابعاد بردارها
- افزایش تعداد ایپوک‌ها
- تنظیم اندازه پنجره برای درک بهتر متن‌ها

✓ بهبود سرعت مدل (Speed Improvement)

- استفاده از چند هسته CPU
- استفاده از Negative Sampling یا Hierarchical Softmax به جای Softmax
- کاهش سایز بردارها برای سرعت بیشتر
- حذف کلمات رایج (Subsampling با ample=1e-5)

- ✓ بهبود نتایج خروجی (Output Quality Improvement)

  - ترکیب CBOW و Skip-gram برای بهبود دقت
  - استفاده از مدل‌های جدیدتر مثل FastText یا GloVe
  - کاهش Overfitting با تکنیک‌های Regularization در مدل‌های پیچیده‌تر

- ✓ بهبود نمایش بردارها (Visualization Improvement)

  - استفاده از تکنیک‌های پیشرفته‌تر مثل t-SNE یا UMAP به جای PCA
  - استفاده از نمایش سه‌بعدی (3D Visualization) برای نمایش بهتر

- ✓ بهبود ارزیابی (Evaluation Improvement)

  - تست مدل با Benchmarks معروف مثل WordSim353 یا Analogies
  - تنظیم دقیق Hyperparameters (window, negative, epochs, vector_size)

۶. سوال شماره ۶:

بطور کلی در این سوال قرار است ۲ روش CBOW و Skip-gram را با دیتاست Text8، پیاده سازی و مقایسه کنیم.

مراحل زیر در این بخش پیاده سازی شده اند:

۱) دانلود و آماده سازی مجموعه داده

مجموعه داده Text8 یک مجموعه داده محبوب برای آموزش مدل‌های جاسازی کلمات (word embeddings) است. این مجموعه از ویکی‌پدیا استخراج شده و شامل حدود ۱۷ میلیون کلمه در یک فایل متنی واحد است که کلمات با فاصله از هم جدا شده‌اند و هیچ نشانه‌گذاری (punctuation) وجود ندارد.

```
# Downloading the Text8 dataset using Gensim's downloader
dataset_path = api.load("text8", return_path=True)

# Display the dataset path
print(f"Dataset downloaded to: {dataset_path}")
```

✓ 0.8s

```
Dataset downloaded to: C:\Users\niush/gensim-data\text8\text8.gz
```

```
# Reading the dataset using Text8Corpus
text8_corpus = Text8Corpus(dataset_path)

# Converting the corpus to a list of sentences
text8_corpus = [list(sentence) for sentence in text8_corpus]
```

✓ 1.5s

- مرحله ۱: دانلود مجموعه داده Text8

از downloader کتابخانه Gensim برای دریافت این مجموعه داده استفاده کردیم. مجموعه داده به‌صورت یک فایل فشرده دانلود می‌شود.

- مرحله ۲: تبدیل به لیستی به نام text8_corpus

در این مرحله، ما فایل دانلود شده را خوانده و آن را به یک لیست از کلمات تبدیل می‌کنیم.

۲) آموزش مدل در مود CBOW

ایجاد و آموزش مدل (با پارامتر های گفته شده):

```python
# Parameters
vector_size = 300   # Dimensionality of the word vectors
window_size = 5     # Number of context words to consider
min_word_count = 5  # Minimum word count threshold
num_workers = 4     # Number of CPU cores to use
epochs = 1          # Number of training epochs

# Initialize and train the Word2Vec model using CBOW (sg=0)
CBOW_model = Word2Vec(
    sentences=text8_corpus,
    vector_size=vector_size,
    window=window_size,
    min_count=min_word_count,
    workers=num_workers,
    sg=0  # 0 for CBOW
)

# Train the model for the specified number of epochs
CBOW_model.train(text8_corpus, total_examples=CBOW_model.corpus_count, epochs=epochs)

# Saving the model
CBOW_model.save("text8_word2vec_cbow.model")

print("Model training completed successfully!")
```

✓ 1m 4.1s

```
Model training completed successfully!
```

CBOW (Continuous Bag of Words) تکنیکی است که در آن مدل، کلمه فعلی را براساس زمینه کلمات اطرافش پیش‌بینی می‌کند.

ما از مدل Word2Vec کتابخانه Gensim استفاده کردیم. یک شیء Word2Vec ایجاد می‌کنیم که پارامتر sg=0 دارد، یعنی مدل برای آموزش با روش **CBOW** تنظیم شده است. سپس از تابع train() برای تعداد مشخصی از epochs استفاده می‌کنیم تا مدل را آموزش دهیم. در نهایت نیز مدل آموزش‌دیده شده برای استفاده‌های بعدی روی دیسک ذخیره می‌شود.

پارامترهای کلیدی:

1. size (dimensions):

   تعداد ابعاد برای وکتورهای کلمات. معمولاً برای داشتن embeddingهای معنادار روی ۳۰۰ تنظیم می‌شود.

2. window (context size):

   حداکثر فاصله بین کلمه فعلی و کلمات همسایه‌اش. مقدار معمولی برای این پارامتر ۵ است.

3. min_count:

   حداقل تعداد تکرار یک کلمه که باید در نظر گرفته شود. اغلب روی ۵ تنظیم می‌شود تا کلمات نادر نادیده گرفته شوند.

4. workers:

   تعداد هسته‌های CPU مورد استفاده. هسته‌های بیشتر = آموزش سریع‌تر. در اینجا از ۴ هسته استفاده خواهیم کرد.

5. sg:

   اگر روی ۰ تنظیم شود یعنی **CBOW** اگر روی ۱ تنظیم شود، مدل به‌صورت **Skip-gram** عمل می‌کند.

6. epochs:

تعداد دفعاتی که مدل به‌طور کامل از روی مجموعه داده عبور می‌کند. طبق دستورالعمل، ما فقط از ۱ epoch استفاده می‌کنیم.

۳) مقایسه شباهت کلمات

```python
# Function to find and print the top 5 most similar words and save their vectors
def print_and_save_similar_words(model, words, dict_vect, file_name, top_n_words=5):

    for word in words:
        if word in model.wv:
            display(Markdown(f"**{word}:**"))

            dict_vect[word] = model.wv[word]

            # Find similar words (this is the main part)
            similar_words = model.wv.most_similar(word, topn=top_n_words)
            # print(type(similar_words))
            # print(len(similar_words))
            # print(similar_words)

            for similar_word, similarity_score in similar_words:
                display(Markdown(f"- {similar_word}: {similarity_score:.4f}"))
                # Save the similar word vectors to the dictionary
                if similar_word not in dict_vect:
                    dict_vect[similar_word] = model.wv[similar_word]

        else:
            display(Markdown(f"**{word}** is not in the vocabulary."))

    # Save the word vectors to a file for future use
    with open(f"{file_name}.pkl", "wb") as file:
        pickle.dump(dict_vect, file)

    return dict_vect
```

حال که مدل Word2Vec را در حالت **CBOW** آموزش داده‌ایم، می‌توانیم ارزیابی کنیم که مدل تا چه اندازه روابط بین کلمات را یاد گرفته است.

یکی از روش‌های این ارزیابی، پیدا کردن کلماتی است که بیشترین شباهت را به یک کلمه مشخص دارند. این شباهت بر اساس **تشابه کسینوسی (cosine similarity)** محاسبه می‌شود.

✅شباهت کسینوسی چیست؟

- شباهت کسینوسی، **زاویه بین دو وکتور کلمه** را اندازه‌گیری می‌کند.
- مقادیر بین **-۱(کاملاً مخالف)** تا **۱ (یکسان)** قرار دارند.
- امتیازهای بالاتر در شباهت نشان‌دهنده این است که کلمات **در زمینه‌های مشابهی به کار رفته‌اند** و معانی مشابهی دارند.

۱. **بارگذاری مدل: (در صورت نیاز)**
   - مدل از فایلی که قبلاً ذخیره کرده‌ایم بارگذاری می‌شود.
۲. **بررسی شباهت کلمات:**
   - برای هر کلمه در لیست داده شده:
     - بررسی می‌کنیم که آیا کلمه در دایره واژگان مدل وجود دارد یا نه.
     - از تابع model.wv.most_similar() برای پیدا کردن **۵ کلمه مشابه‌ترین** استفاده می‌کنیم.
     - کلمه و امتیاز شباهتش را نمایش می‌دهیم.
۳. **مدیریت کلماتی که وجود ندارند:**

- اگر کلمه‌ای در دایره واژگان موجود نباشد، یعنی در هنگام آموزش به حد آستانه‌ی min_count نرسیده است.

نمونه نتیجه:

queen:

- elizabeth: 0.6840
- princess: 0.6837
- victoria: 0.6121
- prince: 0.5875
- king: 0.5872

۴) بصری سازی بردارهای embedding

```python
# Apply PCA to reduce dimensionality from 300 to 2
pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(word_vectors_list_values) # Result

# Plotting the words in 2D space
plt.figure(figsize=(8, 6))
plt.scatter(reduced_vectors[:, 0], reduced_vectors[:, 1])

# Annotate each point with the word
for i, word in enumerate(word_vectors_list_keys):
    plt.annotate(word, (reduced_vectors[i, 0], reduced_vectors[i, 1]), fontsize=12)

plt.title("Fine-Tuned Word Embeddings Visualization using CBOW")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.grid(True)
plt.show()
```

از آنجا که‌embedding های کلمه، وکتورهای با ابعاد بالا هستند (در مورد ما، ۳۰۰ بعدی)، مشاهده‌ی مستقیم آن‌ها دشوار است. برای قابل فهم کردنشان، ما ابعاد آن‌ها را به ۲ بعد کاهش می‌دهیم با استفاده از **تحلیل مؤلفه‌های اصلی.(PCA)**

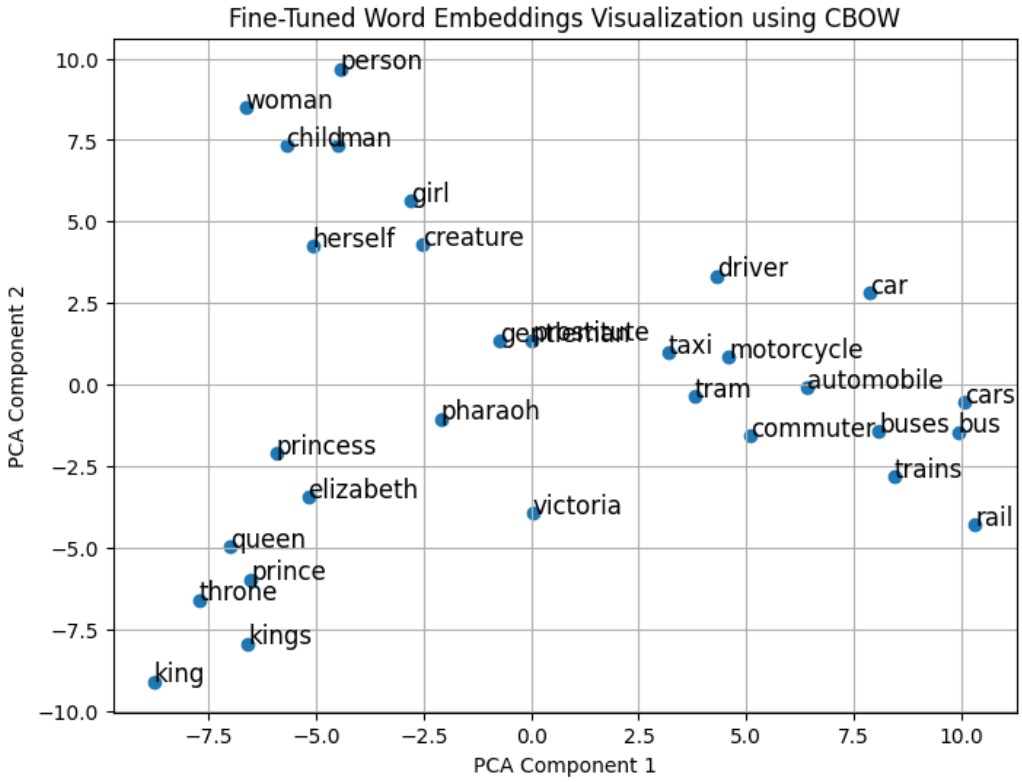
PCAچیست؟

تحلیل مؤلفه‌های اصلی (PCA) تکنیکی است که برای کاهش ابعاد داده‌های بزرگ استفاده می‌شود، در حالی که تا حد ممکن اطلاعات مرتبط حفظ می‌شود.

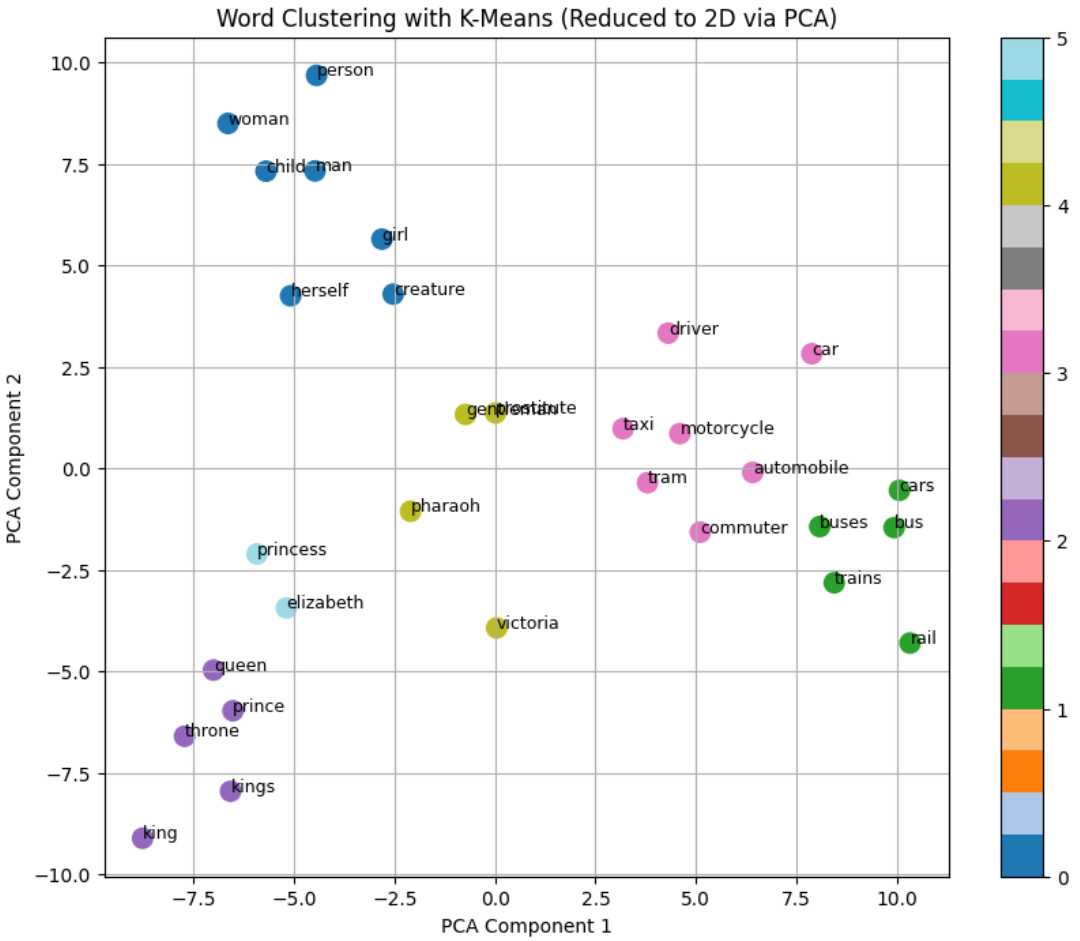
- **ورودی :**وکتورهای ۳۰۰ بعدیembedding های کلمه
- **خروجی :**وکتورهای ۲ بعدی برای رسم و نمایش آسان.
- **هدف :**به‌تصویرکشیدن روابط و گروه‌بندی‌ها بین کلمات.

مراحل تصویری‌سازی Word Embeddings

۱. **استخراج وکتورهای کلمه:**
   - وکتورهای کلمه را برای مجموعه‌ای از کلمات انتخاب‌شده استخراج می‌کنیم.
۲. **اعمال PCA**
   - از PCA برای کاهش ابعاد از **۳۰۰ به ۲** استفاده می‌کنیم.
۳. **رسم کلمات در فضای دوبعدی:**
   - کلمات را در یک فضای دوبعدی ۲ D space نمایش می‌دهیم.



در انتها یک k-means هم برای نمایش بهتر نتایج اعمال کردیم:



به همین تناسب همین مراحل برای روش Skip-gram هم پیاده‌سازی شد.

مقایسه کامل CBOW و Skip-gram

معیار	Skip-gram	CBOW (Continuous Bag of Words)
نوع مدل	مدل پیش‌بینی کلمات زمینه از کلمه هدف	مدل پیش‌بینی کلمه هدف از کلمات زمینه
نحوه آموزش	یادگیری مبتنی بر هدف(Target-based)	یادگیری مبتنی بر زمینه(Context-based)
ورودی مدل	یک کلمه هدف(Target Word)	کلمات زمینه(Context Words)
خروجی مدل	کلمات زمینه(Context Words)	یک کلمه هدف(Target Word)
نوع داده ورودی	تنها یک کلمه	مجموعه‌ای از کلمات (چندین کلمه)
پیش‌بینی می‌کند؟	کلمات زمینه با استفاده از کلمه هدف	کلمه هدف با استفاده از کلمات زمینه
بیشترین کاربرد	داده‌هایی که کلمات نادر دارند(Rare Words)	داده‌های بزرگ) مثل(Wikipedia, Text8
سرعت آموزش	کندتر (به دلیل پیش‌بینی چندین کلمه به‌طور همزمان)	سریع‌تر (به دلیل پیش‌بینی تک کلمه‌ای)
دقت آموزش	بالاتر برای کلمات نادر (بهتر یاد می‌گیرد)	پایین‌تر برای کلمات نادر
حساسیت به داده‌های نادر	قوی (عملکرد بهتر روی کلمات نادر)	ضعیف (عملکرد ضعیف‌تر روی کلمات نادر)
استفاده بهینه از حافظه	ضعیف‌تر (به دلیل نیاز به پردازش بیشتر)	بهتر (به دلیل پردازش سریع‌تر)
کیفیتEmbeddings	مناسب برای کلمات نادر	مناسب برای کلمات پرتکرار
بهترین کاربردها	زمانی که دقت برای کلمات نادر مهم است.	زمانی که سرعت مهم است و داده‌ها بزرگ هستند.
تابع هدف(Loss Function)	Hierarchical SoftmaxیاNegative Sampling	Hierarchical SoftmaxیاNegative Sampling
تعداد نمونه‌های آموزشی	بیشتر (چندین خروجی برای هر ورودی)	کمتر (تک خروجی برای هر ورودی)
زمان آموزش	کندتر ازCBOW	سریع‌تر ازSkip-gram
کیفیت Embeddings نهایی	بسیار بالا برای کلمات نادر	نسبتاً بالا برای کلمات پرتکرار
مقیاس‌پذیری	کمتر مقیاس‌پذیر (زمان و حافظه بیشتری نیاز دارد)	بسیار بالا (برای داده‌های بزرگ مناسب)
پیچیدگی محاسباتی	بیشتر (به‌دلیل استفاده از هدف‌های مختلف)	کمتر) به‌دلیل استفاده از(Context

- اگر داده‌های بزرگ و متنوع است و بیشتر کلمات پرتکرار هستند:

از CBOW بهتر است استفاده کنیم چون سریع‌تر است و به خوبی برای کلمات پرتکرار عمل می‌کند. مثال: آموزش با دادگان بزرگ مثل Wikipedia یا Text8.

- اگر هدف تولید embeddingهای دقیق برای کلمات نادر است:

از Skip-gram بهتر است استفاده کنیم چون برای یادگیری کلمات نادر عملکرد بهتری دارد. مثال: تحلیل زبان‌های کمتر متداول یا داده‌هایی با کلمات خاص و منحصر به فرد.

منابع:

https://jaketae.github.io/study/word2vec/

https://www.youtube.com/watch?v=NXTqdjGd9es

https://en.wikipedia.org/wiki/Distributional_semantics

https://en.wikipedia.org/wiki/Semantic_theory